

Rapport du Premier Travail Pratique – Série 1

**TP 1 : Collecte et prétraitement de données
textuelles par Web Scraping**

Réalisé par :
Zouhair Sakane

Réalisé par :
Zouhair Sakane

Module :

- **Applications de l'Intelligence Artificielle**

Année universitaire :

2025 – 2026

Date de remise :

06/05/2026

Table of Contents

I. Cadre de projet :	3
1. Contexte :	3
2. Objectif :	3
II. Installation des bibliothèques Python :	3
III. Mise en place de l'environnement – Structure des dossiers :	6
IV. Analyse du site cible et stratégie de scraping	9
1. Idée générale d'après le TP :	9
2. Analyse de l'interface visuelle page de liste d'offres et page de détail d'une offre:	10
3. Analyse de La structure HTML du page de liste d'offres et page de détail d'une offre:	14
4. Respect des règles “robots.txt” et planification du scraping des données :	18
1. Respect des règles du fichier robots.txt	18
2. Planification de l'extraction des données	20
IV. Scraping des détails de chaque offer d'après url	21
1. Extraction des détails (fonction principale) :	21
V. prétraitement et analyse des données après scraping	24
1. Nettoyage de donnees :	24
2. Nettoyage de donnees :	25

I. Cadre de projet :

1. Contexte :

Une startup marocaine souhaite analyser les offres d'emploi pour identifier les compétences recherchées, les salaires et les tendances du marché. Ce travail consiste à collecter et prétraiter des données textuelles issues du web.

2. Objectif :

Une startup marocaine souhaite analyser les offres d'emploi pour identifier les compétences recherchées, les salaires et les tendances du marché. Ce travail consiste à collecter et prétraiter des données textuelles issues du web.

II. Installation des bibliothèques Python :

Pour installation des bibliothèques python on va exécuter la commande suivante dans terminal :

```
pip install requests beautifulsoup4 tqdm nltk spacy pandas wordcloud matplotlib
```

Modèle français de spaCy Dans le terminal :

```
python -m spacy download fr_core_news_sm
```

Pour moi je fais ca dans mon code :

```

1  import nltk
2  nltk.download("punkt")
3  nltk.download("punkt_tab")
4  nltk.download("stopwords")

```

Bibliothèque	Pourquoi je l'ai choisie
requests	Permet d'envoyer des requêtes HTTP pour récupérer le contenu HTML des pages du site Emploi.ma durant le web scraping.
BeautifulSoup (bs4)	Utilisée pour analyser le code HTML des pages et extraire les informations importantes comme les offres d'emploi, les compétences et les critères.
time	Sert à ajouter des pauses entre les requêtes afin d'éviter de surcharger le serveur du site web.
random	Permet de générer des délais aléatoires entre les requêtes pour rendre le scraping plus naturel et limiter les blocages.
json	Utilisée pour sauvegarder et lire les données collectées au format JSON.
csv	Permet d'exporter les données des offres d'emploi dans un fichier CSV exploitable avec Excel ou Pandas.
os	Utilisée pour créer et gérer automatiquement les dossiers et chemins de fichiers du projet.

tqdm	Affiche une barre de progression pendant le scraping afin de suivre l'avancement du traitement.
re	Utilisée pour le nettoyage des textes grâce aux expressions régulières (suppression des caractères spéciaux, espaces inutiles, etc.).
nltk	Bibliothèque de traitement automatique du langage naturel utilisée pour la tokenisation et la gestion des stopwords en français.
stopwords (nltk.corpus)	Permet de supprimer les mots fréquents sans importance comme « le », « de », « et ».
word_tokenize (nltk.tokenize)	Sert à découper les textes en mots (tokens) pour préparer les données au traitement NLP.
spaCy	Utilisée pour la lemmatisation des mots afin de réduire les variations linguistiques à leur forme de base.
pandas	Permet de manipuler et analyser facilement les données sous forme de tableaux (DataFrame).
Counter (collections)	Utilisé pour compter les occurrences des compétences et identifier les plus demandées.
WordCloud	Permet de générer un nuage de mots afin de visualiser les termes les plus fréquents dans les offres d'emploi.
matplotlib.pyplot	Utilisée pour afficher et sauvegarder les visualisations graphiques comme le WordCloud.

III. Mise en place de l'environnement – Structure des dossiers :

Afin d'organiser proprement le projet, nous avons adopté l'arborescence suivante :

```
projet_emploi/  
├── scraping/  
│   ├── scrape_emploi.py  
│   └── config.py  
├── data/  
│   ├── brutes/  
│   ├── nettoyees/  
│   └── analyses/  
├── rapports/  
└── requirements.txt
```

Cette séparation permet de distinguer les scripts (***scraping/***), les données brutes et nettoyées (***data/***), ainsi que les résultats d'analyse (***analyses/***). Le fichier requirements.txt liste toutes les bibliothèques nécessaires à la reproductibilité

Pour créer cette structure on va créer par cmd (windows) :

```
Windows PowerShell
PS C:\Users\Zouha\desktop> mkdir projet_emploi

Directory: C:\Users\Zouha\desktop

Mode                LastWriteTime         Length Name
----                -
d-----          06/05/2026   16:08             projet_emploi

PS C:\Users\Zouha\desktop> mkdir projet_emploi\scraping, projet_emploi\data, projet_emploi\rapports

Directory: C:\Users\Zouha\desktop\projet_emploi

Mode                LastWriteTime         Length Name
----                -
d-----          06/05/2026   16:08             scraping
d-----          06/05/2026   16:08             data
d-----          06/05/2026   16:08             rapports

PS C:\Users\Zouha\desktop> mkdir projet_emploi\data\brutes, projet_emploi\data\nettoyees, projet_emploi\data\analysees

Directory: C:\Users\Zouha\desktop\projet_emploi\data

Mode                LastWriteTime         Length Name
----                -
d-----          06/05/2026   16:08             brutes
d-----          06/05/2026   16:08             nettoyees
d-----          06/05/2026   16:08             analysees
```

```
PS C:\Users\Zouha\desktop> New-Item -ItemType File -Path projet_emploi\scraping\scrape_emploi.py -Force
```

```
Directory: C:\Users\Zouha\desktop\projet_emploi\scraping
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	06/05/2026 16:08	0	scrape_emploi.py

```
PS C:\Users\Zouha\desktop> New-Item -ItemType File -Path projet_emploi\scraping\config.py -Force
```

```
Directory: C:\Users\Zouha\desktop\projet_emploi\scraping
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	06/05/2026 16:08	0	config.py

```
PS C:\Users\Zouha\desktop> New-Item -ItemType File -Path projet_emploi\requirements.txt -Force
```

```
Directory: C:\Users\Zouha\desktop\projet_emploi
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	06/05/2026 16:09	0	requirements.txt

```
PS C:\Users\Zouha\desktop> tree projet_emploi /F
```

```
Folder PATH listing
```

```
Volume serial number is 9A7E-C6D8
```

```
C:\USERS\ZOUHA\DESKTOP\PROJET_EMPLOI
```

```
requirements.txt
|
|_ data
|   |_ analysees
|   |_ brutes
|   |_ nettoyes
|_ rapports
|_ scraping
    config.py
    scrape_emploi.py
```


IV. Analyse du site cible et stratégie de scraping

1. Idée générale d'après le TP :

Après avoir créé tous les fichiers, l'étape suivante consiste à analyser le site réel. Cette analyse comprend :

- **Analyse préalable** : identifier structure des URLs, HTML des pages liste et détail.
- **Respect des règles** : consulter robots.txt, respecter les délais et pages interdites.
- **Planification des données** : lister les champs à extraire par offre (Métiers, Secteurs, Études, Expérience, Contrat, Régions, Langues).

2. Analyse de l'interface visuelle page de liste d'offres et page de détail d'une offre:



D'après l'image, nous avons trouvé les critères de l'annonce pour le poste. Nous allons les scraper, mais le problème est le suivant : comment peut-on récupérer le champ « **Métier** » lorsqu'il n'est pas visible dans le frontend et qu'il est stocké directement dans le backend ?

D'après le choix d'une matière sur le site, j'ai remarqué que l'URL change de

<https://www.emploi.ma/recherche-jobs-maroc> vers
https://www.emploi.ma/recherche-jobs-maroc/?f%5B0%5D=im_field_offre_metiers%3A1127.

J'ai aussi utilisé l'extension **Wappalyzer**, et d'après les résultats, le site est développé avec **Drupal (PHP)**. C'est un **CMS** classique où les contenus ne sont pas toujours affichés directement dans le frontend, mais générés dynamiquement à partir d'une base de données.

TECHNOLOGIES

MORE INFO

Export

CMS


[Drupal](#)

Widgets


[AddToAny](#)

Analytics


[Google Analytics](#)

GA4

Security


[reCAPTCHA](#)

Font scripts


[Google Font API](#)

Miscellaneous


[Open Graph](#)


[HTTP/3](#)

Programming languages


[PHP](#)

CDN


[Cloudflare](#)

JavaScript libraries


[jQuery UI](#)

1.10.2


[jQuery](#)

1.12.4

On retrouve donc des éléments comme les offres d'emploi, les critères des annonces et les filtres (comme les métiers), qui sont gérés côté backend via des champs **Drupal** stockés en base de

données (probablement **MySQL** ou **MariaDB**), puis affichés sous forme de pages HTML.

Dans notre cas, les critères de l'annonce pour le poste sont bien visibles dans le frontend, donc faciles à scraper. Par contre, le champ "**Métier**" pose problème, car il n'est pas affiché directement dans le HTML. Il est seulement utilisé comme filtre (**im_field_offre_metiers:1127**) côté backend.

Cela veut dire que le métier est stocké dans la base de données sous forme d'un champ référencé (taxonomy/field Drupal), et qu'il n'apparaît pas directement dans le contenu de la page.

Le vrai problème est donc : comment récupérer le métier si cette information n'est pas affichée dans le frontend, mais seulement utilisée dans les requêtes backend ?

Pour y arriver, il faudrait soit :

- retrouver la correspondance entre l'ID (1127) et son nom de métier via une autre source (API de taxonomy ou page de filtres),
- soit analyser les requêtes internes de Drupal pour identifier où est stockée la relation entre les métiers et leurs IDs.

En résumé : les critères de l'annonce sont directement scrapables depuis le HTML, mais le métier est stocké côté backend sous forme d'ID dans Drupal. Il n'est donc pas visible dans le DOM et nécessite une autre méthode pour être récupéré.

3. Analyse de La structure HTML du page de liste d'offres et page de détail d'une offre:

```

1 <div class="page-search-jobs-wrapper">
2
3 <!-- Pagination -->
4 <div class="pagination">
5 <span>Page :</span>
6 <ul>
7 <li class="pager-current pagination-numbers">1</li>
8 <li class="pager-item active pagination-numbers">
9 <a title="Go to page 2" href="/recherche-jobs-maroc?page=1">2</a>
10 </li>
11 <li class="pager-item active pagination-numbers">
12 <a title="Go to page 3" href="/recherche-jobs-maroc?page=2">3</a>
13 </li>
14 <li class="pager-item breaker pagination-numbers">...</li>
15 <li class="pager-item active pagination-numbers">
16 <a title="Go to last page" href="/recherche-jobs-maroc?page=29">30</a>
17 </li>
18 <li class="pager-next active pagination-next">
19 <a title="Next page" href="/recherche-jobs-maroc?page=1"></a>
20 </li>
21 </ul>
22 </div>
23
24 <!-- Jobs list -->
25 <div class="page-search-jobs-content">
26
27 <!-- JOB CARD -->
28 <div class="card card-job featured" data-href="https://www.emploi.ma/offre-emploi-maroc/ingenieur-process-hf-casablanca-9272501">
29 <picture>
30 <a href="/recruteur/8914043">
31 
32 </a>
33 </picture>
34
35 <div class="card-job-detail">
36 <h3>
37 <a href="/offre-emploi-maroc/ingenieur-process-hf-casablanca-9272501">
38 Ingénieur Process (H/F) - Casablanca
39 </a>
40 </h3>
41
42 <a class="company-name" href="/recruteur/8914043">EXPERT EYE ENGINEERING</a>
43
44 <div class="card-job-description">
45 <p>
46 Nous sommes à la recherche d'un Ingénieur Process (H/F)...
47 </p>
48 </div>
49
50 <ul>
51 <li>Niveau d'études : Bac+5 et plus</li>
52 <li>Expérience : 5 à 10 ans</li>
53 <li>Contrat : CDI</li>
54 <li>Région : Casablanca-Mohammedia</li>
55 <li>Compétences : BTP, Énergie, Tuyauterie</li>
56 </ul>
57
58 <time datetime="2026-05-07">07.05.2026</time>
59 </div>
60 </div>
61
62 <!-- (Other job cards repeat same structure) -->
63
64 </div>
65 </div>

```

D'après code l'extraction de vos données est complexe car la page n'est pas structurée comme un jeu de données propre et organisé.

D'abord, chaque fiche de poste est conçue uniquement pour l'affichage **HTML**, ce qui disperse les informations : le titre se trouve dans une balise **<h3>**, l'entreprise dans un lien **<a>** séparé, la description au milieu d'un long paragraphe **<p>**, et les compétences au sein d'une liste ****. Il faut donc localiser et regrouper manuellement chaque élément.

De plus, la structure n'est pas constante d'une offre à l'autre. Certaines fiches n'affichent pas le nom de l'entreprise ou la ville, ou utilisent des formats différents, ce qui empêche le scraper de s'appuyer sur un modèle fixe et fiable.

Le plus gros obstacle reste certains champs comme le « métier », qui sont totalement absents du HTML. Ils sont stockés côté backend sous forme d'identifiants (comme l'ID 1127) pour la gestion des filtres. Le libellé réel n'apparaît jamais sur la page, ce qui nécessite une source externe pour le décoder.

Enfin, comme le site utilise Drupal, une partie des données est générée dynamiquement depuis une base de données plutôt que d'être directement inscrite dans le code HTML visible.

En résumé : l'information est morcelée entre différentes balises, parfois manquante, et souvent masquée derrière des identifiants techniques au lieu d'être affichée en clair. Cependant, dans la page de détail de chaque offre, j'ai constaté que le code HTML est bien structuré. Mon idée est donc de scraper les données directement depuis chaque offre individuellement.


```
1 <!-- ===== CRITERES DU POSTE ===== -->
2 <section class="job-criteria">
3
4 <h3 class="job-title">
5   Critères de l'annonce pour le poste : Ingénieur Process (H/F) - Casablanca
6 </h3>
7
8 <ul class="arrow-list">
9
10 <li><strong>Métier</strong> : <span>Métiers du BTP</span></li>
11
12 <li><strong>Secteur d'activité</strong> : <span>Ingénierie, études développement</span></li>
13
14 <li><strong>Type de contrat</strong> : <span>CDI</span></li>
15
16 <li><strong>Région</strong> : <span>Casablanca-Mohammedia</span></li>
17
18 <li><strong>Ville</strong> : <span>Casablanca</span></li>
19
20 <li><strong>Travail à distance</strong> : <span>Non</span></li>
21
22 <li>
23   <strong>Niveau d'expérience</strong> :
24   <span>Expérience entre 5 ans et 10 ans - Expérience > 10 ans</span>
25 </li>
26
27 <li><strong>Niveau d'études</strong> : <span>Bac+5 et plus</span></li>
28
29 <li><strong>Nombre de poste(s)</strong> : <span>1</span></li>
30
31 <li><strong>Management d'équipe</strong> : <span>Non</span></li>
32
33 </ul>
34
35 <ul class="skills">
36 <li>Faisabilité</li>
37 <li>APS</li>
38 <li>APD</li>
39 <li>Ingénierie de base et détaillée</li>
40 <li>HAZID</li>
41 <li>HAZOP</li>
42 <li>ATFX</li>
43 <li>Energie</li>
44 <li>Tuyauterie</li>
45 <li>Dimensionnement</li>
46 <li>Instrumentation</li>
47 <li>BTP</li>
48 </ul>
49
50 </section>
51
52 <!-- ===== RESUME + ENTREPRISE ===== -->
53 <div class="container-flex">
54
55 <!-- ===== RESUME DU POSTE ===== -->
56 <div class="card card-block card-block-summary">
57
58 <h2 class="card-block-title">Résumé du poste</h2>
59
60 <div class="card-block-content">
61
62 <ul>
63
64 <li>Métier : Métiers du BTP</li>
65
66 <li>Localisation : Casablanca Mohammedia</li>
67
68 <li>Télétravail : Non</li>
69
70 <li>Expérience : 5 à 10 ans et plus</li>
71
72 <li>Niveau d'études : Bac+5 et plus</li>
73
74 <li>Contrat : CDI</li>
75 </ul>
76
77 <div class="block-links">
78 <a href="#" form="express-application" class="btn btn-primary">
79   Postuler
80 </a>
81 </div>
82 </div>
83 </div>
84
85 <!-- ===== ENTREPRISE ===== -->
86 <div class="card card-block card-block-summary">
87
88 <h2 class="card-block-title">Entreprise</h2>
89
90 <div class="card-block-company">
91
92 <ul>
93
94 <li>
95 <h3>
96 <a href="/recruteur/8914043">
97   EXPERT EYE ENGINEERING
98 </a>
99 </h3>
100 </li>
101
102 <li>
103 Secteur d'activité : Ingénierie, études développement
104 </li>
105
106 <li>
107 Site Internet :
108 <a href="https://experteeye.ma/" target="_blank" rel="nofollow">
109   experteeye.ma
110 </a>
111 </li>
112
113 <li>
114 Offres d'emploi :
115 <a href="/recherche-jobs-maroc?fq[0]=is_recruiter_nid:8914043">
116   Voir toutes les annonces
117 </a>
118 </li>
119
120 <li>
121 Description :
122 With over 20 years of experience, EXPERT EYE ENGINEERING is a trusted engineering firm.
123 </li>
124 </ul>
125
126 
127
128 </div>
129 </div>
130 </div>
131 </div>
132
133 </div>
```

4. Respect des règles “robots.txt” et planification du scraping des données :

1. Respect des règles du fichier robots.txt

D’après le fichier robots.txt du site, on observe les règles suivantes :

- Le site autorise l’exploration (crawl) pour tous les robots :
 - User-agent: *
 - Content-Signal: search=yes → cela signifie que l’extraction de contenu pour construire un index de recherche est autorisée.
- Le site interdit certaines utilisations :
 - ai-train=no → il est interdit d’utiliser le contenu pour entraîner ou fine-tuner des modèles d’IA.
 - Certains bots IA sont bloqués directement (GPTBot, CCBot, ClaudeBot, etc.).
- ai-input n’est pas explicitement autorisé ni interdit donc cette utilisation n’est pas accordée par défaut.

Conclusion

On peut faire du scraping classique pour la recherche et l’analyse (search=yes), mais on ne doit pas utiliser les données pour entraîner des modèles d’IA (ai-train=no).

La mention dans le robots.txt :

“ANY RESTRICTIONS EXPRESSED VIA CONTENT
SIGNALS ARE EXPRESS RESERVATIONS OF RIGHTS

UNDER ARTICLE 4 OF THE EUROPEAN UNION
DIRECTIVE 2019/790...”

signifie que le site utilise la directive européenne 2019/790
sur le droit d’auteur (Copyright in the Digital Single Market).

Lien avec le texte marocain (loi 09-08)

Même si cette directive est européenne, elle peut être
comparée au cadre juridique marocain, notamment la **loi
n° 09-08 relative à la protection des données
personnelles** publiée dans le *Bulletin Officiel n° 5714
(2009)*.

Cette loi marocaine :

- protège les **données personnelles des individus**
- encadre le **traitement des données informatiques**
- impose que toute collecte ou traitement de données
respecte la **vie privée et le consentement**

Correspondance entre les deux cadres

- La directive européenne 2019/790 protège les **droits des
producteurs de contenu numérique** (copyright et
utilisation des données)
- La loi marocaine 09-08 protège les **données personnelles
des utilisateurs**

Conclusion

Les deux textes vont dans la même logique :

- limiter l’exploitation non autorisée des données

- protéger les droits des personnes ou des propriétaires de contenu
 - encadrer l'utilisation des données dans un contexte numérique et automatisé (scraping, IA, etc.)
- Ainsi, la mention dans le robots.txt est une **réservation de droits légale**, et elle est cohérente avec les principes de protection des données et du contenu présents dans la loi marocaine 09-08.

2. Planification de l'extraction des données

Pour chaque offre d'emploi, l'objectif est d'extraire les informations suivantes :

- URL de l'offre : lien vers la page du poste
- Titre : intitulé du poste
- Entreprise : nom de la société
- Description : texte descriptif du poste
- Métier : domaine professionnel (ex: BTP, informatique...)
- Secteur : secteur d'activité de l'entreprise
- Contrat : type de contrat (CDI, CDD...)
- Région : région de l'offre
- Ville : ville du poste
- Télétravail : présence ou non du travail à distance
- Expérience : niveau d'expérience demandé
- Niveau d'étude : diplôme requis
- Compétences : liste des skills demandées
- Date de publication : date de mise en ligne de l'offre

IV. Scraping des détails de chaque offer d'après url

1. Extraction des détails (fonction principale) :

```
1  def scrape_job(url):
2
3      soup = get_soup(url)
4      if not soup:
5          return None
6
7      # TITLE
8      title = soup.select_one("h1")
9      title = clean(title.get_text()) if title else ""
10
11     # COMPANY
12     company = soup.select_one(".card-block-company h3 a")
13     company = clean(company.get_text()) if company else ""
14
15     # DESCRIPTION
16     desc = soup.select_one(".job-description")
17     desc = clean(desc.get_text(" ")) if desc else ""
18
19     # CRITERIA
20     criteria = extract_criteria(soup)
21
22     # SKILLS
23     skills = [
24         clean(li.get_text())
25         for li in soup.select("ul.skills li")
26     ]
27
28     # =====
29     # PUBLICATION DATE (NEW)
30     # =====
31     pub_date = ""
32     date_box = soup.select_one(".page-application-details")
33
34     if date_box:
35         p = date_box.find("p")
36         if p:
37             pub_date = format_date(p.get_text())
38
39
40     return {
41         "url": url,
42         "titre": title,
43         "entreprise": company,
44         "description": desc,
45
46         "metier": criteria["metier"],
47         "secteur": criteria["secteur"],
48         "contrat": criteria["contrat"],
49         "region": criteria["region"],
50         "ville": criteria["ville"],
51         "teletravail": criteria["teletravail"],
52         "experience": criteria["experience"],
53         "niveau_etude": criteria["etude"],
54
55         "competences": skills,
56         "date_publication": pub_date
57     }
58
59
```

Ce que fait cette fonction :

- Télécharge la page de détail
- Extrait toutes les informations d'une offre
- Gère les champs manquants
- Retourne un dictionnaire structure
- Je envoie une requête HTTP
- je transforme la page en BeautifulSoup
- Si erreur alors retourner "None"
- Tu utilises CSS selectors pour récupérer les blocs HTML.
- Je utilises CSS selectors pour récupérer les blocs HTML.
- En fin retourner Chaque offre ressemble dans le dossier data/brutes/ soit json or csv comme ici


```
{  
  "url": "...",  
  "titre": "...",  
  "entreprise": "...",  
  "description": "...",  
  "metier": "...",  
  "secteur": "...",  
  "contrat": "...",  
  "region": "...",  
  "ville": "...",  
  "teletravail": "...",  
  "experience": "...",  
  "niveau_etude": "...",  
  "competences": [],  
  "date_publication": "2026-05-07"  
}
```

V. prétraitement et analyse des données après scraping

1. Nettoyage de données :

Dans le fichier cleaner.py du module de scraping, j'ai réalisé un processus de prétraitement des données.

Tout d'abord, le **nettoyage du texte** est effectué grâce à la fonction clean_text(), qui permet de :

- supprimer les balises HTML
- convertir le texte en minuscules
- éliminer la ponctuation ainsi que les espaces inutiles

Ensuite, un **traitement NLP** est appliqué via la fonction `preprocess()`, comprenant :

- la tokenisation avec `word_tokenize`
- la suppression des stopwords
- la lemmatisation avec `spaCy`

Le résultat obtenu est une liste de mots normalisés et exploitables.

Enfin, la fonction `process_dataset()` applique ce traitement à l'ensemble du dataset en :

- chargeant le fichier `jobs.json`
- ajoutant un champ "tokens" à chaque offre (basé sur la description)
- sauvegardant le résultat dans `jobs_clean.json`

2. Nettoyage de donnees :

Dans cette étape, nous réalisons une analyse exploratoire des données issues du scraping. Les compétences les plus fréquentes sont extraites à partir du champ `competences`, tandis que les villes les plus représentées sont calculées à l'aide de `pandas`. Enfin, un nuage de mots (`WordCloud`) est généré à partir des tokens obtenus lors du prétraitement afin de visualiser les tendances lexicales des offres d'emploi.

et results est :

```
ville
Casablanca      288
Rabat           97
Meknès          42
Marrakech       30
Mohammedia      24
/ Travail En Remote 22
/ Travail à Distance 20
Tanger          17
Agadir          11
Nouaceur        9
Name: count, dtype: int64
```



Une figure est créée avec Matplotlib afin de visualiser les résultats de l'analyse. Le WordCloud est affiché dans cette figure, puis sauvegardé sous forme d'image dans le dossier data/analysees.